Assignment #3: Iterator pattern & Classes relationships

Due Date 1: Wednesday, November 4, 2026, 5:00 pm EST
Due Date 2: Wednesday, November 11, 2026, 5:00 pm EST

Learning objectives:

- C++ classes, constructors, destructors, and operations
- Iterator pattern
- Class composition, aggregation, and specialization

- **Questions 1a, and 3a are due on Due Date 1; questions 1b, 2, and 3b are due on Due Date 2.**
- On this and subsequent assignments, you will take responsibility for your own testing. This assignment is designed to get you into the habit of thinking about testing *before* you start writing your program. For each question, you will be given a compiled executable program which is a program representing a solution to each question. You should use these provided executables to help you write your test cases, as they can show you the resultant output for given inputs. If you look at the deliverables and their due dates, you will notice that there is *no* C++ code due on Due Date 1. Instead, you will be asked to submit test suites for C++ programs that you will later submit by Due Date 2.

  Test suites will be in a format compatible with that presented in class.
- Design your test suites with care; they are your primary tool for verifying the correctness of your code. Note that test suite submission `zip` files are restricted to contain a maximum of 40 tests. The size of each input `(.in)` file is also restricted to 300 bytes, and each output file `(.out)` is restricted to 1,000 bytes. This is to encourage you not to combine all of your testing in a single pair of files.
- You must use the standard C++ I/O streaming and memory management (MM) facilities on this assignment; you may **not** use C-style I/O or MM. More concretely, you may `import` the following C++ libraries (and no others!) for the current assignment: `iostream`, `fstream`, `sstream`, `iomanip`, `string`, and `utility`. MarkUs will be setup to **reject** submissions that use C-style I/O or MM, or libraries other than the ones specified above.
- We will manually check that you follow a reasonable standard of documentation and style, and to verify any assignment requirements that are not automatically enforced by MarkUs. Code to a standard that you would expect from someone else if you had to maintain their code. Further comments on coding guidelines can be found here: `https://cmsweb.utsc.utoronto.ca/cscb07f26/codingGuidelines/`
- We have provided some code and sample executables in the subdirectory `codeForStudents` under the appropriate subdirectories. **These executables have been compiled on the Mathlab server and will not run anywhere else.**
- **You may not ask public questions on Piazza about what the programs that make up the assignment are supposed to do.** A major part of this assignment involves designing test cases, and questions that ask what the programs should do in one case, or another will give away potential test cases to the rest of the class. Questions found in violation of this rule will be marked private or deleted; repeat offences could be subject to discipline.
- You may publicly discuss the coding assessment portion of the questions on Piazza as long as solutions are neither discussed nor revealed.

Question 1

1. **(DD1: 20%, DD2: 20%)**A markup language is a system for annotating documents. In markup languages such as HTML and XML, annotations consist of tags, identified by the starting and end characters `< >`. Tags can further contain attributes or children tags/text and are closed by `</ >`. For example, in the following excerpt, `document`, `body`, and `text` are tags. The `text` tag contains attributes `id` and `value`. Currently the first text tag has `"t1"` as the `id`, and `"CSCB07"` as the `value` attribute.

```
<document>
  <body>
    <text id='t1' value='CSCB07'>
    </text>
    <text id='t2' value='CSCB09'>
    </text>
    <text id='t3' value='CSCC01'>
    </text>
  </body>
</document>
```

In this exercise, you will write a class to manage tags in a document. Our markup language is loosely inspired by HTML and XML, but it is quite simplified. Our tags can be of any type (such as `document`, `body`, or `text`) and can optionally contain the attributes `id` and `value` (and no other). Additionally, they can contain any number of children tags. There is always a top-level `document` tag. The example document above is a valid document in our markup language and also an example of the formatting that should be used by the program when printing the tags.

We have provided the declaration of the `Tag` class in the file `tag.cc` in the `a3/q1` directory. **You may not change the contents of `tag.cc` other than by adding private instance members and comments, i.e., the public interface must stay exactly the same.**

```
class Tag {
    std::string type;   // the type of the tag
    std::string id;     // the id attribute of the tag
    std::string value;  // the value attribute of the tag
    Tag *parent;        // pointer to the parent tag (or nullptr for the document
                         // tag, which does not have a parent)
    Tag **children;     // array of children tag pointers
    int childrenLength;   // current number of children tags
    int childrenCapacity; // current capacity of the children array
    // public methods; please check tag.cc for details
};
```

The diagram in Figure 1 shows an overview of the tags composing the example document and the pointers to their respective parent and children tags.

Your job is to implement all the methods of the `Tag` and `TagIterator` classes in the file `tag-impl.cc`. Please read carefully the documentation for each method in `tag.cc`. The comments explain what they must do.

Implementation details:

- The `children` array contains the children Tags. When a Tag is created, initialize `children` to `nullptr`. Initialize `childrenLength` and `childrenCapacity` to zero. When the first child Tag is added, create an array in the heap with size one. You must keep track of the current capacity (size) of the array in `childrenCapacity` and the current number of children tags (positions used in the array) in `childrenLength`. You will increase the capacity of the array using a doubling strategy. Each time a

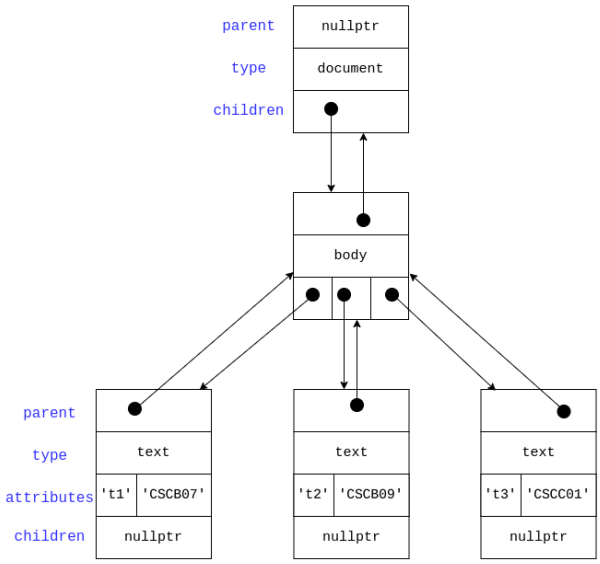


Figure 1:

new child Tag is added, if the length of the array is already equal to the capacity, you must double the current capacity of the array (i.e., create a new array with double the capacity of the current one and move all the children Tags over to the new array). When children tags are removed from the array, you do not need to reduce the capacity.

- Because we test your submission by comparing the output of your program with the expected output of the correct program, you will lose marks if your `Tag::print()` does not precisely match the output of our implementation. You can use the document provided at the start of the question as an example of what the formatted output should look like. Note that each tag start markup and tag end markup should be printed on a new line. Children tags must be indented with two spaces per level. We are also providing you a sample executable that produces the correct output. Please test the provided executable for every potential type of output and ensure that your solutions matches the sample output precisely (use an utility like `diff` to ensure that the outputs are identical; don't rely only on a visual inspection). The `produceOutputs` and `runSuite` scripts can help you do that.
- We also provided the declaration of `TagIterator`, which is a implementation of the `Iterator` design pattern for the `children` array. You must also implement the methods of this class. Used together with the methods `Tag::begin()` and `Tag::end()`, it can be used by a client to iterate through all the children of a tag.
- You can implement the `operator<<` for Tags (also declared in `tag.cc`) by just calling the tag's `print` method.
- **Your program must not leak any memory.**

A test harness is provided in `main.cc` (which uses `controller.cc` and `controller-impl.cc`). You can use it to test your `Tag` class. **The test harness is not robust, and you are not to devise tests for it, just for the `Tag` class. Do not change these files.** We have also provided a sample executable (`a3q1`) that you can use for tests. Both files can be found in the directory `a3/q1`.

The test harness supports the following commands. Please read the contents of `main.cc`, `controller.cc`, and `controller-impl.cc` if you need more details about them. Note that the harness keeps track of the "current" tag at any moment and most of the commands operate on it. Also, note that the harness automatically creates a top-level `document` tag. You can set its attributes and add children to it. **However, you cannot delete, cut, or copy the `document` tag and you should not create new tags of this type.**

Command	Description
`print`	Prints the current tag to the standard output.
`add` *type*	Creates a new tag of the *type* specified by the argument, adds it as a child of the current tag, and makes it the new current tag. Note that *type* cannot be `"document"`.
`delete`	Deletes the current tag, removes it from its parent, and makes its parent the new current tag.
`parent`	Sets the parent of the current tag as the new current tag.
`up`	Sets the topmost tag (`<document>`) as the current tag.
`id` *val*	Sets the `id` attribute of the current tag to *val*. Note that *val* cannot contain whitespaces.
`value` *val*	Sets the `value` attribute of the current tag to *val*. Note that *val* cannot contain whitespaces.
`find` *type*	Finds the first tag that is a child of the current tag and has the specified type. If such a tag is found, it is set as the current tag. Otherwise, the message `<`*type*`> not found.` is written to `cerr` and the current tag does not change.
`findId` *id*	Finds the first tag that is a child of the current tag and has the specified id. If such a tag is found, it is set as the current tag. Otherwise, the message `Tag with id '`*id*`' not found.` is written to `cerr` and the current tag does not change.
`list`	Lists the type of the direct children of the current tag.
`cut`	Removes the current tag from its parent and moves it to the program's internal clipboard. Sets its parent tag as the current tag.
`copy`	Copies the current tag to the program's internal clipboard.
`paste`	Adds a new child to the current tag, whose contents are a copy of the tag currently in the clipboard, and updates the pointers of the child and current tag accordingly. The new child becomes the new current tag.
`quit`	Quits the program.

Submission:

(a) **Due on Due Date 1:** Design the test suite `suiteq1.txt` for this program and zip the suite into `a3q1a.zip`.

(b) **Due on Due Date 2:** Implement this in C++ and place the files `tag.cc` and `tag-impl.cc` in the zip file, `a3q1b.zip`. We will provide the `main.cc`, `controller.cc`, and `controller-impl.cc` to test your submission.

Question 2

2. **(DD2: 10%)**In this question, you will extend your solution from Question 1 to support these two additional commands:

Command	Description
`next`	• If the current tag is not the last child tag in its parent, then set the next sibling (i.e., the tag that comes right after the current one in its parent's children array) as the current tag in the `controller` and call `Controller::printCurrent()` to print the current status to `cout`. • If the current tag is already the last child tag in its parent, just print `"Current tag is already the last child of its parent."` to `cerr` and do nothing else. • If the current tag is the top-level tag (i.e., it does not have a parent), just print `"Current tag is the upper level tag."` to `cerr` and do nothing else.
`previous`	• If the current tag is not the first child tag in its parent, then set the previous sibling (i.e., the tag that comes right before the current one in its parent's children array) as the current tag in the `controller` and call `Controller::printCurrent()` to print the current status to `cout`. • If the current tag is already the first child tag in its parent, just print `"Current tag is already the first child of its parent."` to `cerr` and do nothing else. • If the current tag is the top-level tag (i.e., it does not have a parent), just print `"Current tag is the upper level tag."` to `cerr` and do nothing else.

The diagram in Figure 2 shows the changes of current tag when executing commands `next` and `previous`.

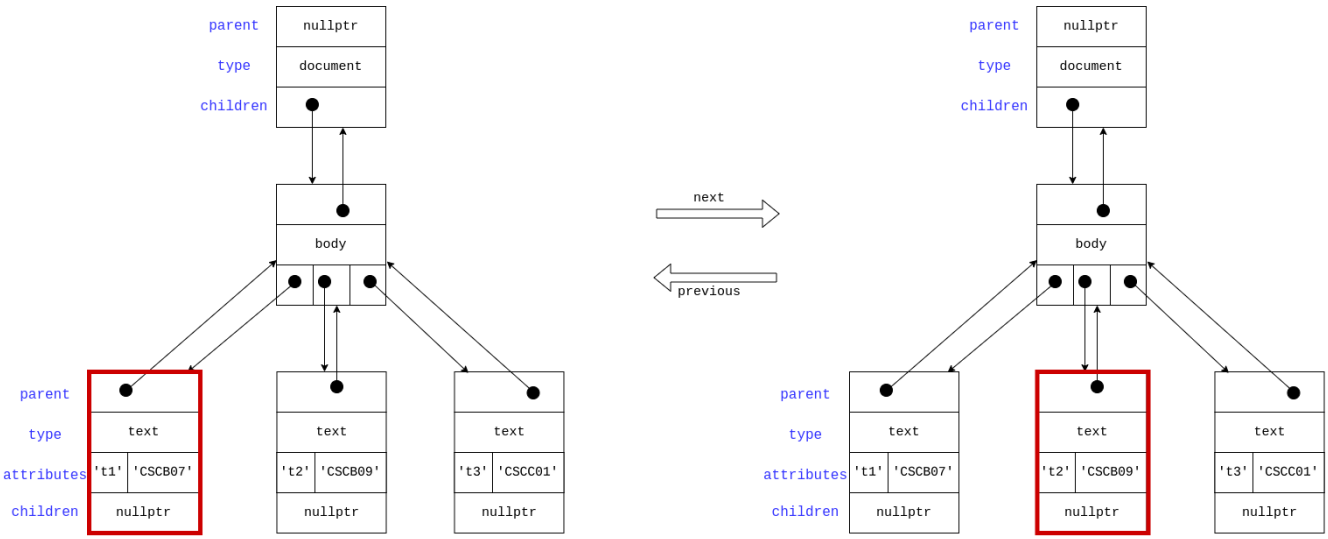


Figure 2:

You are not given any starting code for it. Therefore, you should use your solution from Question 1 as the starting code. For this question, you are allowed to modify any file in the solution. But please note that your modifications must follow the C++ and object-oriented programming guidelines taught in this course.

You will need to modify the `main()` function to support the two additional commands and make any other changes necessary to the program to make them work. You must ensure that the other commands of the test harness continue working as before, i.e., the only change in the program's behaviour is that two new commands are now supported. Your updated solution must also not leak any memory.

We have provided a sample executable (`a3q2`) that you can use for tests. The sample executable can be found in the directory `a3/q2`.

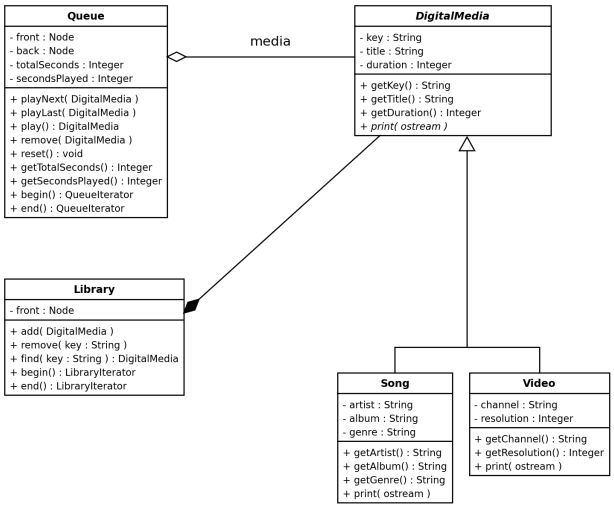
Submission:

(a) **Due on Due Date 1:** Nothing.

(b) **Due on Due Date 2:** Implement this in C++ and place **all** the files necessary to compile your program in `a3q2b.zip`.

Question 3

3. **For this question, you will be using `#include`, header (.h) files, and a provided Makefile. More details below.**
(DD1: 20%, DD2: 30%) This question simulates a simplified version of the play queue in a media streaming service. There is an underlying library of digital media (each item is either a song or a video) and a single queue of the media that are going to be played. Media from the library can be added to the queue to *play next* (the front of the queue) or to *play last* (the back of the queue). In particular, we'll use the Iterator design pattern to traverse the library and the queue objects. The resulting UML class model looks like the following, though you can change your private data field declaration names, and add private methods as desired.



The `Song` and `Video` classes are concrete subclasses of the abstract base class `DigitalMedia`. They just contain data, with the appropriate accessors and no mutators. (One simplifying assumption that you may make is that the keys are alphanumeric sequences.) They implement the pure virtual method `print`, which is used to help the iterators output the object information. Use the provided `util.h` and `util.cc` to simplify your implementations of the associated input operators.

- The `Song` constructor sets the key, title, duration (in seconds), artist, album, and genre information. See the provided starter file for the list of the error messages produced to stderr, and under what order and condition they are output.
- `operator<<` for `Song` outputs the object's information as `(key, "title", "album", "artist", duration, "genre")` where each string that might have whitespace within it (i.e. title, album, artist, genre) is surrounded with double-quotation marks.
- `operator>>` for `Song` reads in information in the following format `key\ntitle\nduration\nartist\nalbum\ngenre\n`. See the provided starter file for the list of the error messages produced to stderr, and under what order and condition they are printed. It sets the `failbit` of the stream, and leaves the object unchanged, if an error occurs.
- The `Video` constructor sets the key, title, duration (in seconds), channel, and resolution (the number of vertical pixels, e.g. 1080) information. See the provided starter file for the list of the error messages produced to stderr, and under what order and condition they are printed.
- `operator<<` for `Video` outputs the object's information as `(key, "title", "channel", duration, resolutionp)` where the strings that might have whitespace within them (i.e. title, channel) are surrounded with double-quotation marks, and the resolution is followed by the letter `p` (e.g. `1080p`).

- `operator>>` for `Video` reads in information in the following format `key\ntitle\nduration\nchannel\nresolution\n`. See the provided starter file for the list of the error messages produced to stderr, and under what order and condition they are printed. It sets the `failbit` of the stream, and leaves the object unchanged, if an error occurs.

The `Library` holds the pointers to the objects of the `DigitalMedia` subclasses, `Song` and `Video`, and owns them. When traversed, the items are output in lexicographical order by their key values i.e. standard `std::string` sort order. The keys must be unique. Implement the library as a singly linked list that is kept in key order.

- `Library::add` adds the specified object pointer to the library, indexed by its key. It will output to stderr the message "`key KKKK already exists in library`" if the key `KKKK` is a duplicate. In that case the item is not stored, and it is deleted since the library is the owner.
- `Library::find` returns the pointer to the object with the specified key, or `nullptr`.
- `Library::remove` removes and deletes the item with the specified key value if it exists. If it doesn't exist, nothing happens, and no error messages are produced.
- `operator<<` outputs the string `Library:\n`, followed by each item in the library in key order. Each item is preceded by a tab character, `'\t'`, and terminated by a newline.
- `operator>>` adds to the existing library by reading a sequence of digital media objects. Each set of input starts with either an `'s'` to denote a `Song`, or a `'v'` to denote a `Video`. The information is then read in according to the specified input format for that class. (The simplest approach is to use the defined input operators to read them into temporary objects whose contents are over-written, and then use copy operations.) Reading stops at EOF, or if an item cannot be read in (nothing is added for it), or if the type is neither `'s'` nor `'v'`, in which case `invalid media type` followed by the character that was read is written to `std::cerr`. In the last two cases the `failbit` of the stream is set.

A `Queue` holds a collection of `DigitalMedia` pointers to the subclasses, in the order in which they will be played: the item at the front is the next one to be played. The queue does not own the items, and the same item may appear in it more than once.

- `Queue::playNext` adds an item to the front of the queue, and `Queue::playLast` adds an item to the back.
- `Queue::play` "plays" the item at the front of the queue, meaning that it removes the item from the queue, adds its duration to the number of seconds played so far, and returns a pointer to it, or `nullptr` if the queue is empty. (It doesn't delete the item, since the queue is not the owner.)
- `Queue::remove` removes every occurrence of the item with the specified address, if there are any. If there aren't, nothing happens and no error messages are produced. (It doesn't delete the item, since the queue is not the owner.)
- `Queue::getTotalSeconds` returns the sum of the durations of the items in the queue.
- `Queue::getSecondsPlayed` returns the number of seconds played so far.
- `Queue::reset` sets that number back to 0.
- `operator<<` for `Queue` uses an iterator to output each item on its own line: a tab character, the item's position in the queue (starting at 0, width 3, padded with '0'), a colon and a space, the item's information, and `starts in N seconds`, where `N` is the iterator's elapsed time at that item. The last line is `Total: N seconds`, where `N` is the total duration of the items in the queue.

Traversing the queue is simulated by iterating over it. When the `QueueIterator` is created, it starts at the first item in the queue, and the amount of elapsed time is 0 seconds. When the iterator is moved via `operator++`, the duration of the current item is added to the elapsed time before the iterator is moved to the subsequent item. The elapsed time at an item is therefore how long it will be before that item starts playing. When the end of the queue is reached, the iterator stops and the elapsed time equals the sum of all of the item durations. Any change to the queue structure invalidates the iterator.

The test harness is not intended to be particularly robust, so do not write test cases for it. See the provided test harness for a description of the commands.

Your program must follow the Iterator design pattern, and **must not leak memory**.

Building your program: the starter files include a file called `Makefile`. A Makefile is a text file that tells the program `make` how to build your program, like which files to compile, with which compiler options, and how to link the results into the executable `a3q3`. When you run `make`, only the files that have changed since the last build are recompiled, so rebuilding is fast. Put your `.h` and `.cc` files (including `main.cc`) and the `Makefile` in the same directory, and in that directory:

- run `make` to compile your program; this creates the executable `a3q3`;
- run `make clean` to delete the object files and the executable, so that the next `make` rebuilds everything from scratch.

You do not have to modify the `Makefile`. A new `.cc` file that you create there is compiled and linked automatically, and when you change a `.h` file, the `.cc` files that include it are recompiled.

a) **Due on Due Date 1:** Design a test suite for this program. Submit a file called `a3q3a.zip` that contains all of the `.txt`, `.in` and `.out` files for the test suite you designed.

b) **Due on Due Date 2**: Write the program in C++. Save your solution in a file named `a3q3b.zip`. It must contain all of your `.h` and `.cc` files, including `main.cc`.

Submission

The following files are due at Due Date 1: `a3q1a.zip`, `a3q3a.zip`.
The following files are due at Due Date 2: `a3q1b.zip`, `a3q2b.zip`, `a3q3b.zip`.